Partial inverses by Sweeping

Jan de Leeuw

March 21, 2016

Contents

1 Simple Sweep **1**

2 Example Sweep **2**

2.1 Using Sweeping for the Inverse 2

2.2 Using Sweeping for Regression 3

2.3 The partial inverse 3

3 Timing **4**

4 Code **5**

4.1 R Code 5

4.2 C code 6

5 NEWS **6**

References **6**

1 Simple Sweep

Suppose A is a square non-singular matrix. A *simple sweep* on the kth element replaces A by the matrix with elements

$$
\begin{aligned}
a_{kk} &= \frac{1}{a_{kk}}, \\
a_{ki} &= -\frac{a_{ki}}{a_{kk}}, \\
a_{ik} &= \frac{a_{ik}}{a_{kk}}, \\
a_{ij} &= a_{ij} - \frac{a_{ik}a_{kj}}{a_{kk}}
\end{aligned}
$$

for $j \neq k$ and $i \neq k$.

In R that becomes

```r
sweep <- function (a, k) {
  p <- a [k, k]
  a [k, k] <- 1 /p
  a [-k, -k] <- a [-k, -k] - outer (a [-k, k], a [k, -k]) / p
  a [k, -k] <- -a[k, -k] / p
  a [-k, k] <- a[-k, k] / p
  return (a)
}
```

2 Example Sweep

```r
set.seed(12345)
mprint (a <- matrix (sample(1:9), 3, 3))
```

```
##      [,1]  [,2]  [,3]
## [1,]  7.00  9.00  4.00
## [2,]  8.00  3.00  2.00
## [3,]  6.00  1.00  5.00
```

```r
mprint (sweep(a, 1))
```

```
##      [,1]  [,2]  [,3]
## [1,]  0.14 -1.29 -0.57
## [2,]  1.14 -7.29 -2.57
## [3,]  0.86 -6.71  1.57
```

2.1 Using Sweeping for the Inverse

```r
sweepSubset <- function (a, k, verbose = FALSE) {
  for (i in k) {
    a <- sweep (a, i)
    if (verbose) mprint (a)
  }
  return (a)
}
```

```r
mprint (b <- cSweep (a, 1 : nrow (a)))
```

```
##      [,1]  [,2]  [,3]
## [1,]  0.05  0.16  0.03
## [2,]  0.16  0.08 -0.09
## [3,]  0.03 -0.09 -0.26
```

```r
mprint (a %*% b)
```

```
##      [,1]  [,2]  [,3]
## [1,]  1.87  1.47 -1.71
## [2,]  0.91  1.33 -0.59
## [3,]  0.57  0.56 -1.23
```

2.2 Using Sweeping for Regression

```
x <- matrix (rnorm (20), 10, 2)
y <- rnorm (10)
mprint (b<-qr.solve (x, y))
```

```
## [1]  0.11 -0.07
```

```
mprint (sum((y-x%*%b) ^ 2))
```

```
## [1]  6.39
```

```
h <- crossprod (cbind (x,y))
mprint (sweepSubset (h, 1 : 2, verbose = TRUE))
```

```
##                y
##    0.05  0.03 -0.11
##   -0.03 13.31 -0.95
## y  0.11 -0.95  6.46
##                y
##    0.05  0.00 -0.11
##    0.00  0.08  0.07
## y  0.11 -0.07  6.39
##                y
##    0.05  0.00 -0.11
##    0.00  0.08  0.07
## y  0.11 -0.07  6.39
```

2.3 The partial inverse

Suppose k is a subset of $\{1, 2, \cdots, n\}$ and $\overline{k}$ is the complimentary subset. In our examples we usually take, for convenience, $k = \{1, 2, \cdots, k\}$. The *partial inverse* of A on k, which generalizes the simple sweep, is

$$\mathbf{inv}_k(A) = \begin{bmatrix} A_{kk}^{-1} & -A_{kk}^{-1}A_{k\overline{k}} \\ A_{\overline{k}k}A_{kk}^{-1} & A_{\overline{k}\overline{k}} - A_{\overline{k}k}A_{kk}^{-1}A_{k\overline{k}} \end{bmatrix}.$$

Why is this called a partial inverse ? If

$$\begin{bmatrix} A_{kk} & A_{k\overline{k}} \\ A_{\overline{k}k} & A_{\overline{k}\overline{k}} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} u \\ v \end{bmatrix},$$

then

$$\begin{bmatrix} A_{kk}^{-1} & -A_{kk}^{-1}A_{k\overline{k}} \\ A_{\overline{k}k}A_{kk}^{-1} & A_{\overline{k}\overline{k}} - A_{\overline{k}k}A_{kk}^{-1}A_{k\overline{k}} \end{bmatrix} \begin{bmatrix} u \\ y \end{bmatrix} = \begin{bmatrix} x \\ v \end{bmatrix},$$

And conversely. Of course all this assumes that A_{kk} is non-singular. The following results are true for an arbitrary index set k and its complement $\overline{k}$.

1. $A = \mathbf{inv}_k(\mathbf{inv}_k(A))$
2. $\mathbf{inv}_k(A) = (\mathbf{inv}_{\overline{k}}(A))^{-1}$
3. $\mathbf{inv}_{\overline{k}}(\mathbf{inv}_k(A)) = A^{-1}$
4. $\mathbf{inv}_\ell(\mathbf{inv}_k(A)) = \mathbf{inv}_k(\mathbf{inv}_\ell(A))$
5. $\mathbf{inv}_k(A) = \mathbf{inv}_{\overline{k}}(A^{-1})$
6. $(\mathbf{inv}_k(A))^{-1} = \mathbf{inv}_k(A^{-1})$

3 Timing

```r
set.seed(12345)
b <- crossprod(matrix(rnorm(1000000),1000,1000))/1000
```

```
## [1] "element sweeping in pure R"
##    user  system elapsed
##  14.974   1.510  15.770
##    user  system elapsed
##  14.399   1.377  14.976
##    user  system elapsed
##  14.334   1.359  14.881
##    user  system elapsed
##  14.388   1.360  14.951
##    user  system elapsed
##  14.362   1.363  14.928
## [1] "element sweeping in R and fortran from R"
##    user  system elapsed
##   1.423   0.310   1.736
##    user  system elapsed
##   1.378   0.301   1.684
##    user  system elapsed
##   1.244   0.348   1.595
##    user  system elapsed
##   1.309   0.262   1.575
##    user  system elapsed
##   1.263   0.267   1.534
## [1] "element sweeping in R and C from R and fortran from C"
##    user  system elapsed
##   0.453   0.011   0.468
##    user  system elapsed
##   0.420   0.009   0.431
##    user  system elapsed
##   0.416   0.011   0.429
##    user  system elapsed
##   0.426   0.011   0.439
##    user  system elapsed
##   0.435   0.011   0.449
## [1] "just solve it"
##    user  system elapsed
##   0.211   0.006   0.071
##    user  system elapsed
##   0.197   0.004   0.071
##    user  system elapsed
##   0.205   0.003   0.069
##    user  system elapsed
##   0.206   0.003   0.070
##    user  system elapsed
##   0.197   0.003   0.070
```

4 Code

4.1 R Code

```
rSweep <- function (a, indi) {
for (j in indi) {
    pv <- a[j, j]
    if (pv == 0) next ()
    pr <- a[j, -j]
    pc <- a[-j, j]
    a[j, j] <- -1 / pv
    a[j, -j] <- pr / pv
    a[-j, j] <- pc / pv
    a[-j, -j] <- a[-j, -j] - outer(pc, pr) / pv
    }
return (a)
}

fSweep <- function (b , ind, eps = 1e-10, trianin = FALSE, trianout = FALSE) {
    if (trianin) {
        m <- length (b)
        n <- as.integer ( (sqrt (1 + 8 * m) - 1) / 2)
        a <- b
    } else {
        n <- nrow (b)
        m <- (n * n + n) / 2
        a <- b[upper.tri (b, diag = TRUE)]
    }
    for (k in ind) {
        a <- .Fortran("gsweep",
            diago = as.double (a[cumsum (1:n)]),
            trian = as.double (a),
            k = as.integer (k),
            l = as.integer (0),
            m = as.integer (m),
            n = as.integer (n),
            e = as.double (eps),
            ifault = as.integer (0))$trian
        }
    if (trianout) {
        return (a)
    } else {
        return (tri2sym (a))
    }
}

cSweep <- function (b , ind, eps = 1e-10, trianin = FALSE, trianout = FALSE) {
    if (trianin) {
        m <- length (b)
        n <- as.integer ( (sqrt (1 + 8 * m) - 1) / 2)
        a <- b
    } else {
```

```r
        n <- nrow (b)
        m <- (n * n + n) / 2
        a <- b[upper.tri (b, diag = TRUE)]
    }
    a <- .C("hsweep",
        diago = as.double (a[cumsum (1:n)]),
        trian = as.double (a),
        n = as.integer (n),
        m = as.integer (m),
        ind = as.integer (ind),
        npiv = as.integer (length (ind)),
        eps = as.double (eps))$trian
    if (trianout) {
        return (a)
    } else {
        return (tri2sym (a))
    }
}

tri2sym <- function (a) {
    m <- length (a)
    n <- as.integer ( (sqrt (1 + 8 * m) - 1) / 2)
    b <- matrix (0, n, n)
    b[upper.tri (b, diag = TRUE)] <- a
    b <- b + t(b)
    diag (b) <- diag (b) / 2
    return (b)
}
```

4.2 C code

```c
void gsweep_(double *, double *, int *, int *, int *, int *, double *, int *);

void
hsweep(double *s, double *trian, int *n, int *m, int *ind, int *np, double *eps) {
    int ifault = 0, l = 0;
    for (int i = 0; i < *np; i++) {
        gsweep_ (s, trian, ind + i, &l, m, n, eps, &ifault);
    }
```

5 NEWS

References